**linked_list.py**

```
1
2  class Node:
3      def __init__(self, data):
4          self.data = data
5          self.next = None
6
7  def iterate_linked_list(head):
8      if head is None:
9          return
10     p = head
11     while p is not None:
12         print(p.data, end=" ")
13         p = p.next
14     print()
15
16 def find_last_node(head):
17     if head is None:
18         return None
19     p = head
20     while p.next is not None:
21         p = p.next
22     return p
23
24 def find_second_last_node(head):
25     if head is None:
26         return None
27     if head.next is None:
28         return None
29     p = q = head
30     while p.next is not None:
31         q = p
32         p = p.next
33     return q
34
35 def find_kth_from_end(head, k):
36     if head is None:
37         return None
38     p = head
39
40     while k != 1 and p.next is not None:
41         p = p.next
42         k -= 1
43
44     if p.next is None and k != 1:
45         return None
46
47     q = head
48     while p.next is not None:
49         p = p.next
50         q = q.next
51
```

```
    return q

def make_last_node_first(head):
    if head is None:
        return None
    if head.next is None:
        return head

    p = q = head
    while p.next is not None:
        q = p
        p = p.next

    p.next = head
    q.next = None
    head = p
    return head

def reverse_linked_list(head):
    if head is None:
        return None
    if head.next is None:
        return head

    p = head
    q = r = None

    while p is not None:
        r = q
        q = p
        p = p.next
        q.next = r

    head = q
    return head

def find_middle(head):
    if head is None:
        return None
    p = q = head

    while q is not None and q.next is not None:
        q = q.next.next
        p = p.next

    return p

def insert_at_beginning(head, x):
    p = Node(x)
    p.next = head
    head = p
    return head

def insert_at_end(head, x):
```

```
    p = Node(x)
    if head is None:
        head = p
        return head

    q = head
    while q.next is not None:
        q = q.next

    q.next = p
    return head

def insert_before_y(head, y, x):
    if head is None:
        return None

    new_node = Node(x)
    p = head
    q = None

    while p.next is not None and p.data != y:
        q = p
        p = p.next

    if p.data != y:
        return head

    q.next = new_node
    new_node.next = p
    return head

def delete_from_end(head):
    if head is None:
        return None
    p = head
    q = None

    while p.next is not None:
        q = p
        p = p.next

    q.next = None
    del p
    return head

def delete_x(head, x):
    if head is None:
        return None
    p = head
    q = None

    while p.next is not None and p.data != x:
        q = p
        p = p.next
```

```
160
161     if p.data != x:
162         return head
163
164     q.next = p.next
165     del p
166     return head
167
168 def main():
169     head = Node(0)
170     a = Node(10)
171     b = Node(20)
172
173     head.next = a
174     a.next = b
175
176     print("list: ", end="")
177     iterate_linked_list(head)
178
179     head = insert_at_beginning(head, 5)
180     print("list: ", end="")
181     iterate_linked_list(head)
182
183 if __name__ == "__main__":
184     main()
185
186 # output:
187 """
188 list: 0 10 20
189 list: 5 0 10 20
190 """
```